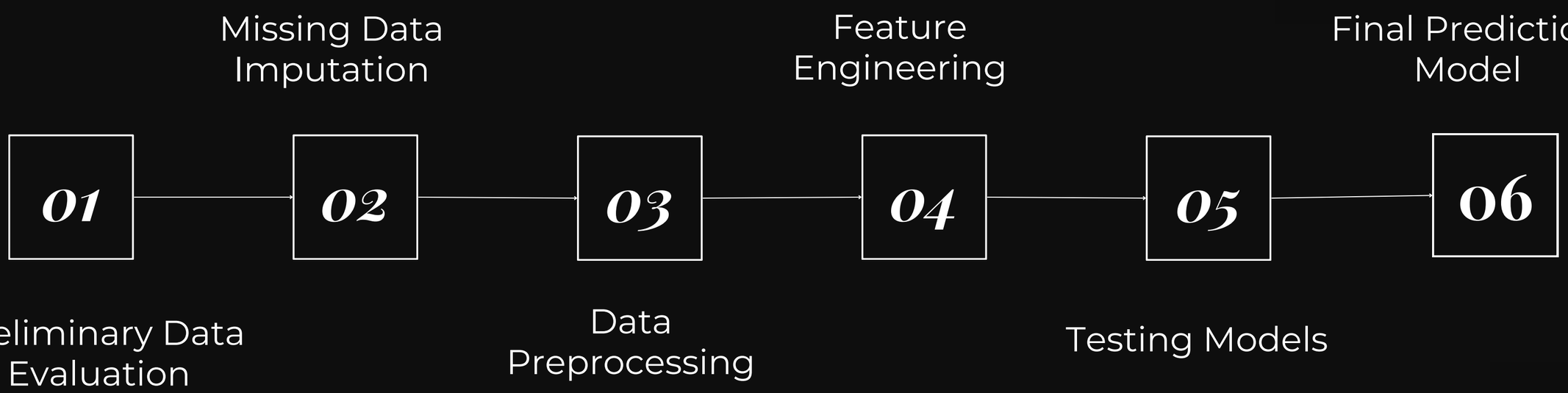
# Finnovation

Team Chimps

# Approach for Task 01



# Preliminary Data Evaluation

Shape of training dataset:
327741 rows x 139 columns

| | |
|---|---|
| **Demographic & Behavioral** | • Age, Vintage, KYC Score<br>• Locker Holder, UID/KYC/INB/eKYC Flags<br>• Income Band |
| **Account & Transactional Data** | • Account Age, Credit Limit, Outstanding Amount<br>• Loan Tenure, Instalment Amount, SI Flag<br>• Monthly Trends of Credit (CR), Debit (SDR), Outstanding Balance<br>• Average MTD, QTD, YTD |
| **Loan Portfolio Features** | • Number of Loans, Loan Limits & Outstandings<br>• IRAC Class (Max), Loan Tenure Progression<br>• Product Type, Aggregated Loan Groupings |
| **Risk History** | • NPA & Risk Grade Tenure (RG1–RG4)<br>• NPA Count, Years Since NPA<br>• IRAC Transitions: Slips, Upgrades<br>• Recent Risk Grades (1 to 3 Years) |
| **Credit Bureau Data** | • CRIF Scores: CRIFF_11 to CRIFF_66<br>• Account & Loan Summary: Active/Overdue Accounts, Balances, Sanctions<br>• Credit Behavior: Inquiries, Delinquencies, Account Age, Credit History |

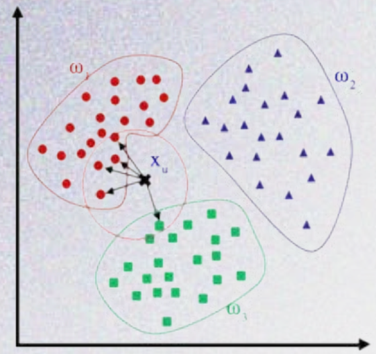
# Missing Data Imputation

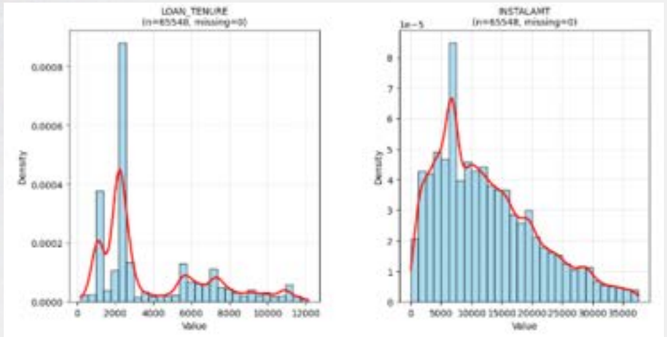
**Category 1: Columns with <0.5% missing values**
-We simply **dropped the rows** with the missing values

**Category 2: Columns with 1-10% missing values**
-We used **KNN Imputation** to fill in those values for Numerical Columns
-Reasons for using this method

- It leverages similar data points (neighbors) to estimate missing values.
- Suitable for cases where mean/median fails to capture local structure.
- Non-parametric and robust to skewness and outliers.
- We used k=5 as a reasonable trade-off between local representation and stability.

-We used **top 3/5 mode imputation** for categorical columns maintaining the initial distribution.

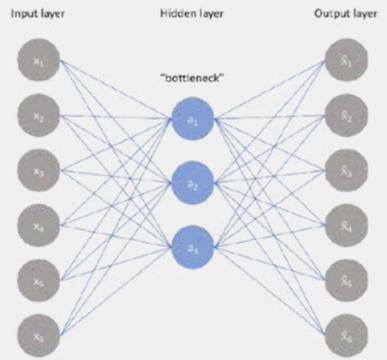


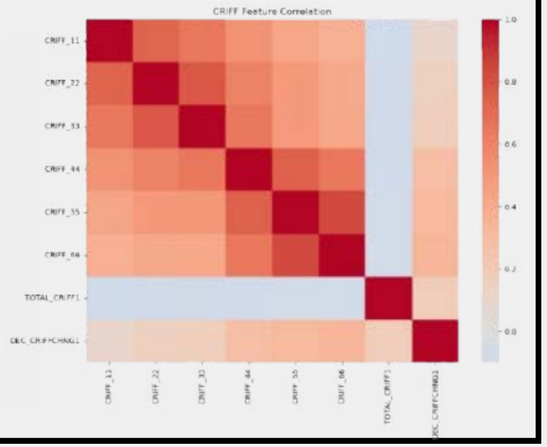




**Category 3: Columns with 10-20% missing values**
-3 columns fell in this range namely CRIFF_11, CRIFF_22, CRIFF_44
-We followed a **hierarchical method for imputation** after observing the heat map.

- First trained an **autoencoder** to reconstruct CRIFF_22 from CRIFF_11
- Then used reconstructed 11 + 22 to estimate CRIFF_33 using a **multi layered perceptron**. Same approach was followed for imputing CRIFF_44 using CRIFF_55 and CRIFF_66

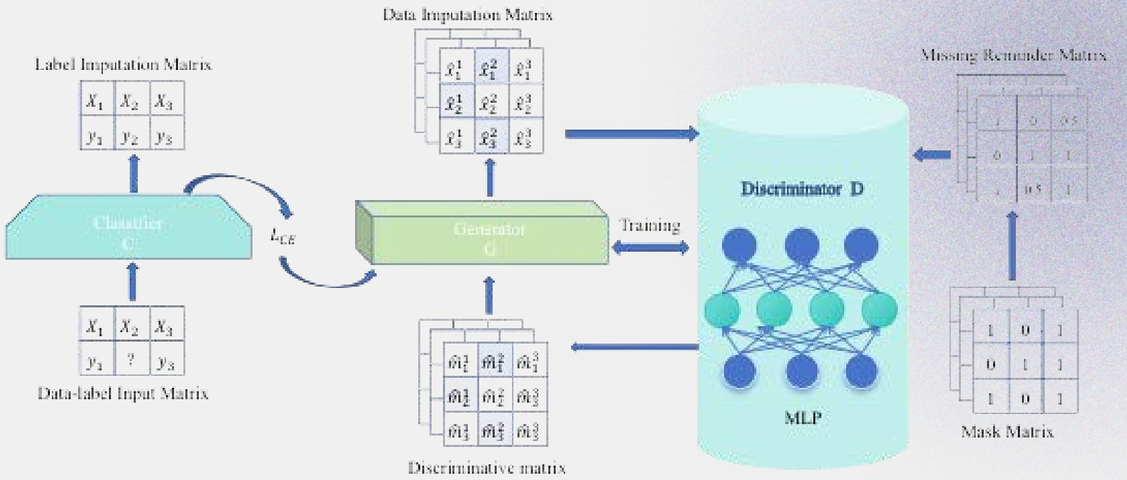
-Autoencoders and MLPs were used because they are capable of learning non-linear inter-feature dependencies, which is not the case for multi variate regression.



**Category 4: Columns with missing values in the range 50-80% (6 columns) and >90% (1 column)**

- We initially considered dropping these columns, but further analysis revealed their **strong relevance to NPAs**. Key columns included: LAST_1_YR_RG4, LAST_3_YR_RG4, FIRST_NPA_TENURE, CUST_NO_OF_TIMES_NPA, LATEST_NPA_TENURE, NO_YRS_NPA, and NO_ENQ.
- We used semi-supervised learning to impute missing values for these, specifically we implemented this paper **SEGAN: A Semi-Supervised Learning Method for Missing Data Imputation.**
- SEGAN imputes missing values using a generator-discriminator setup, combining adversarial learning with self-supervised reconstruction to model complex feature dependencies




The objective function of the SEGAN model can be defined as a minimax game function:

$$\min_{G,C}\max_{D} U(G,D,C) = \mathbb{E}_{\hat{\mathbf{X}},\mathbf{M},\mathbf{R}}\left[\mathbf{M}\odot\log D\left(\hat{\mathbf{X}},\mathbf{R}\right)+(1-\mathbf{M})\odot\log(1-D\left(\hat{\mathbf{X}},\mathbf{R}\right))\right]$$
$$\beta L_{\mathrm{CE}}(\mathbf{y},C(\mathbf{X}))$$
$$= V(G,D)+\beta L(G,C) \tag{11}$$

$$V(G,D) = \int_{\mathcal{X}}\int_{\mathcal{R}}\sum_{i=1}^{n}\sum_{j=1}^{d}\left(\sum_{\mathbf{M}\in\mathbf{M}_1^{\mathbf{ji}}}\log D(\mathbf{X},\mathbf{R})_i^j + \sum_{\mathbf{M}\in\mathbf{M}_0^{\mathbf{ji}}}\log(1-D(\mathbf{X},\mathbf{R})_i^j)\right)p(\mathbf{X},\mathbf{M},\mathbf{R})\,d\mathbf{R}\,d\mathbf{X}$$

# Data Preprocessing

-We applied a **Box-Cox transformation** to the numerical columns to address high skewness.

$$y_i^{(\lambda)} = \begin{cases} \dfrac{y_i^{\lambda}-1}{\lambda} & \text{if } \lambda \neq 0, \\ \ln(y_i) & \text{if } \lambda = 0, \end{cases}$$

Formula for box-cox transformation on positive data

-For categorical columns, we used **label encoding and one-hot encoding** which coniderably increased the number of columns to 750

We used **KL divergence** to assess imputation accuracy for all the columns, with a **worst score of 0.8**—expected for real-world data.

$$D_{\mathrm{KL}}(p(x)\,||\,q(x)) = \sum_{x\in X} p(x)\ln\frac{p(x)}{q(x)}$$

# Feature Engineering

## Clustering Approach

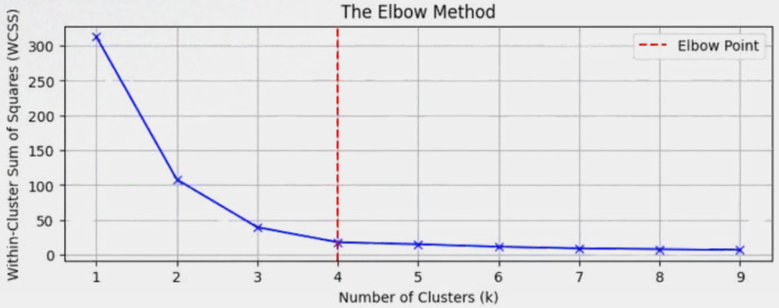
1. **Feature Categorization & Clustering**
   - Segregated 137 features into 5 logical categories:
   - Applied Elbow Method per category to find optimal clusters:
   - **Account:4 | Loan:6 | Risk:4 | Credit:3 | Demographic:2**
2. **Advanced Cluster Features**

Derived 3 key features per cluster:

- **Cluster ID:** Segment assignment
- **Centroid Distance:** Normalized distance from cluster center
- **Cluster Risk:** Default rate within cluster
- **Risk_k** = (Defaulters in k) / (Total members in k)

3. **Cluster Pathway Concept**
   - Each customer mapped to a **5D path** (e.g., [2,4,1,3,2])
   - Used a simple logistic regression on this to get a threshold for final defaulter classification but did not get a good accuracy (**87% with F1 score for minority class as 27%**).
   - So, we dropped this approach but added this as a feature.

| Feature Name | Formula |
|---|---|
| NET_FLOW_TREND | Slope of the linear regression for the net flow series: $\text{slope}\left(\{CR_i - DR_i\}_{i=1}^{12}\right)$ |
| AVG_CREDIT_EMA | Exponential moving average of credit transactions: $\text{EMA}_\alpha(CR_1, CR_2, \ldots, CR_{12})$ |
| AVG_DEBIT_EMA | Exponential moving average of debit transactions: $\text{EMA}_\alpha(DR_1, DR_2, \ldots, DR_{12})$ |
| AVG_NET_FLOW_EMA | Exponential moving average of net flow: $\text{EMA}_\alpha(CR_i - DR_i), \quad \forall i \in \{1, \ldots, 12\}$ |
| BALANCE_VOLATILITY | Standard deviation of balance: $\sigma_{\text{BAL}} = \sqrt{\frac{1}{12}\sum_{i=1}^{12}(BAL_i - \mu_{\text{BAL}})^2}$ |
| QTD_VOLATILITY | Standard deviation of quarterly totals: $\sigma_{\text{QTD}} = \sqrt{\frac{1}{4}\sum_{j=1}^{4}(QTD_j - \mu_{\text{QTD}})^2}$ |
| YTD_VOLATILITY | Standard deviation of yearly totals: $\sigma_{\text{YTD}} = \sqrt{\frac{1}{12}\sum_{i=1}^{12}(YTD_i - \mu_{\text{YTD}})^2}$ |
| UTILIZATION_RATIO | Loan utilization ratio: $\text{UR} = \frac{\text{ALL_LON_OUTS}}{\max(\text{ALL_LON_LIMIT}, \varepsilon)}$ |
| where $\varepsilon = 10^{-6}$ prevents division by zero | |
| RISK_DETERIORATION_RATE | Risk change indicator: $\text{RDR} = \frac{\text{TIMES_IRAC_SLIP}}{\text{TIMES_IRAC_UPR} + \varepsilon}$ |
| with $\varepsilon = 1$ for smoothing | |







We have also **dropped** the Monthly features (12 months) like

- Average Monthly Credit Amount
- Average Monthly Debit Amount
- Average QTD, YTD Balances

# Testing Models

To address the class imbalance in our training dataset, we applied **SMOTE (Synthetic Minority Over-sampling Technique)**. This approach increased the number of minority class samples from **28,113 to 168,678**, significantly improving the class distribution.

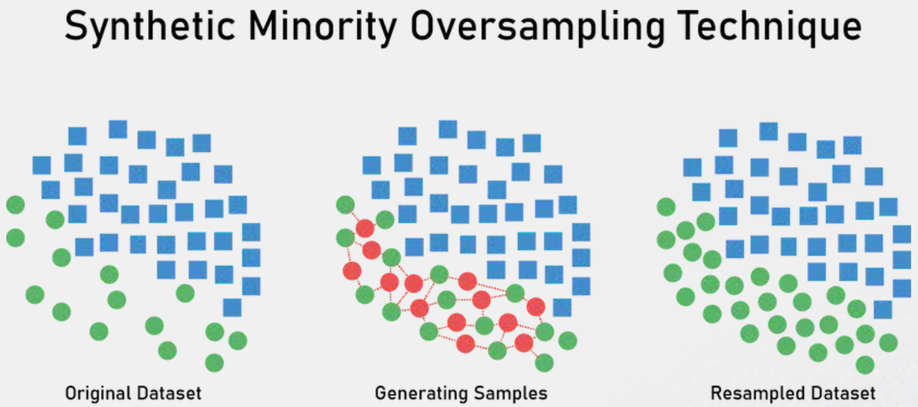
How SMOTE Works:

- Instead of duplicating existing samples, **SMOTE generates synthetic data for the minority class.**
- For each minority class instance, it:
  - **Identifies one of its nearest neighbors** in feature space.
  - **Interpolates** between the two points to create a new, realistic sample along the line connecting them.

This method enriches the training set with more diverse examples of the minority class, leading to better generalization and improved performance in some of our models.

```
Original class distribution:
TARGET
0    232553
1     28113
```

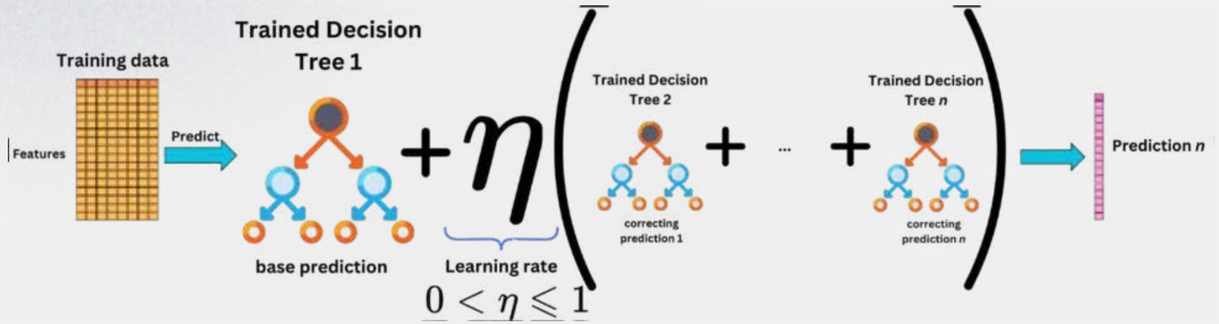
```
After manual SMOTE class distribution:
Class 1: 168678
Class 0: 232553
```



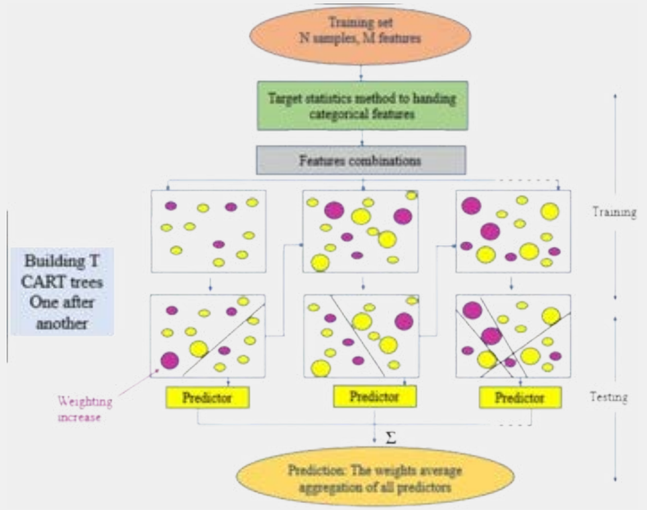

| Model | Accuracy | Average F1 | Average Recall | Average Precision |
|---|---|---|---|---|
| Decision Tree | 0.74 | 0.61 | 0.75 | 0.61 |
| CatBoost | 0.90 | 0.71 | 0.85 | 0.68 |
| Logistic Regression | 0.74 | 0.61 | 0.73 | 0.61 |
| XGBoost with SMOTE | 0.93 | 0.77 | 0.72 | 0.86 |
| Random Forest Classifier | 0.83 | 0.67 | 0.75 | 0.65 |
| Gradient Boosting Classifier on SMOTE | 0.91 | 0.66 | 0.62 | 0.80 |
| Neural Networks/MLPs | 0.78 | 0.61 | 0.69 | 0.60 |

# Final Prediction Model

We implemented a **weighted ensemble** of the CatBoost (trained on the normal dataset) and XGBoost (trained on the SMOTE dataset). The weights for the ensemble of each of them were determined by applying the **softmax function on their avg F1 score**.

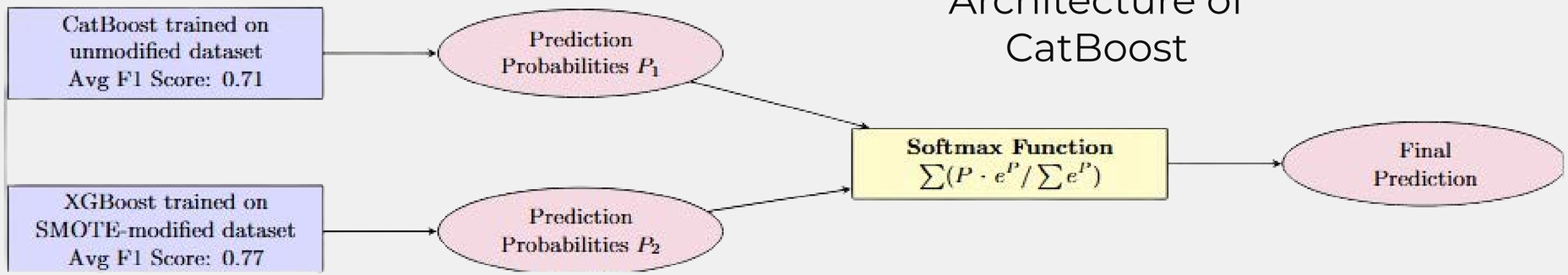



Architecture of XGBoost




Architecture of CatBoost

Overall Prediction Pipeline



# Approach for Task 02



# Phases for Task 2



## Google Dorking

- We use SerpAPI with our own API key instead of manual site:/filetype: dorks.
- Under the hood, search_person(name, phone, serpapi_key) fans out concurrent requests to:

1) LinkedInScraper (in webfinder/linkedin.py)
2) InstagramScraper (in webfinder/instagram.py)
3) GitHubScraper (in webfinder/github.py)
4) PDFScraper (in webfinder/pdf_scraper.py)

**Some examples for google dork commands:**

- site:instagram.com "Abhi Tanwar" "+91" (location OR city OR lives OR from)
- site:linkedin.com/in "Abhi Tanwar" "+91"
- filetype:pdf "Abhi Tanwar" "+91"
- site:github.com "Abhi Tanwar" location
- filetype:txt "Abhi Tanwar" "+91" (India OR Delhi)
- filetype:doc "Abhi Tanwar" "+91" (bio OR contact OR resume)

# Phases for Task 2

```
You are a location analysis expert. I will provide you with data collected from Google dorking
searches for a specific individual, along with additional demographic information.

**Input Data Includes:**
- Social media profiles and posts (LinkedIn, Instagram, Facebook)
- Public documents and PDFs
- Biographical information and contact details
- Location tags, check-ins, and any publicly available data
- **Age of the individual**
- **City tier classification** (Tier 1/2/3 city background)
Your task is to:
1. **Analyze all provided data** for location indicators while considering:
   - Age-appropriate locations (educational institutions, career stage, lifestyle patterns)
   - City tier context (migration patterns from smaller to larger cities, typical career progression)
   - Explicit location mentions in posts/profiles
   - Workplace or educational institution addresses
   - Geotagged photos, check-ins, and time zone patterns
   - Local references, landmarks, businesses mentioned
   - Phone number area codes or regional identifiers
2. **Apply demographic reasoning:**
   - Use age to filter relevant educational institutions, career levels, and lifestyle indicators
   - Consider typical migration patterns based on city tier (rural to urban, smaller city to metropolitan areas)
   - Factor in age-appropriate social activities and location preferences
3. **Predict the most likely current location** including:
   - Primary residence (city/area) with tier classification
   - Confidence level for each prediction
4. **Provide detailed reasoning** explaining how age and city tier background
 influenced your location analysis and final predictions.
```

## Extracting Features

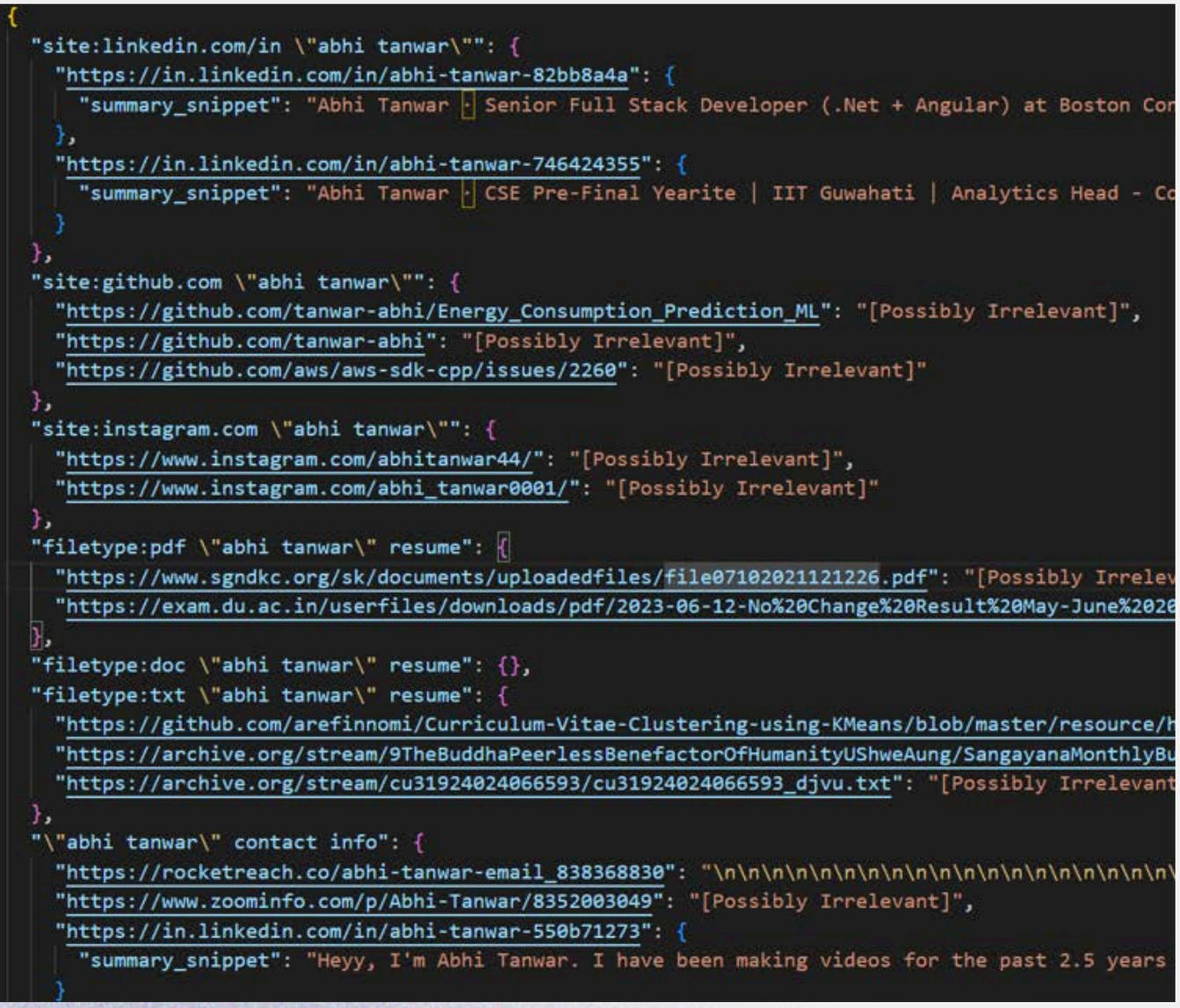
- Single Prompt to Mistral-7B-Instruct

We frame a system + user prompt that instructs the model to parse each URL's summary_snippet and emit a structured JSON with fields: names, profile_urls, organizations, degrees, job_titles, locations.

- Light Post-Processing Checks

Filter out entries marked "[Possibly Irrelevant]".
Ensure each profile_url maps to at least one extracted field; flag empties for manual review.

# Phases for Task 2




1. **Rich, Structured Results**

- **Grouped by source query** (e.g. LinkedIn, GitHub, Instagram) for instant platform visibility
- **Profiles paired with concise summary** snippets (e.g. “CSE Pre-Final Yearite | IIT Guwahati…”)
- **“[Possibly Irrelevant]”** flags let us immediately discard noise

2. **Automation & Traceability**

- **JSON can be fed directly** into our pipeline—no more manual link-copying
- Every scrape is timestamped and cached, creating a full audit trail
- Consistent, repeatable data dumps enable easy A/B testing and regression checks

3. **Seamless Downstream Integration**

- Prompt-ready structure: inline this JSON into Mistral-7B without extra parsing
- Use snippet-vs-irrelevant flags as simple features in our ensemble model
- Instantly spot missing domains or low-coverage areas and auto-trigger retries

# Walkthrough



## Input Given

Name : Abhi Tanwar
Mobile Number : 9876543210



## Google Dorking

Links produced like:
https://www.linkedin.com/in/abhi-tanwar-746424355/
https://www.instagram.com/abhitanwar281/



## Data Extraction

Extracts keywords like IITG, Meghalaya, CSE, C&A.



## Location Clues

From bio and post location mentions, tries to extract names of city.



## LLM Feature Extraction

Feed the raw SerpAPI JSON (profile URLs + summary_snippets) into Mistral-7B-Instruct.
Model emits a structured JSON with keys:
locations (e.g. ["Guwahati, Assam"])
organizations (e.g. ["IIT Guwahati"])
degrees (e.g. ["B.Tech Computer Science"])
job_titles (e.g. ["Analytics Head"])



## Final Location with Confidence Ratio

Final Location : IIT Guwahati, Assam
Confidence Ratio:0.88

# Fallback Model- Data Generation

**Core Idea:** We created a **fake dataset** that simulates the data a **bank could extract** from UPI or Net Banking **transactions** for a user. It includes the **timestamp, transaction amount, IP address, and user ID** for each transaction.

## IP Address

Every device has **two types** of IP addresses:
**Private IP Address**:

- Assigned within a **local network** (e.g., home Wi-Fi).
- **Not visible** to external servers or the internet.

**Public IP Address:**

- Reveals **approximate geographical location** (city-level).
- Public IP of a transaction **can be recorded** at the bank's server side when a user performs **UPI or Net Banking** transactions over the internet.

## Data Generated

- Users were **classified into three categories** based on **travel frequency**: Frequent, Moderate, and Less frequent travellers.
- A dummy mapping of **IP addresses** for **200 Indian cities** was created.
- **1000 users** were simulated over July 2025 (**~90-95 transactions/user**).
- The columns of the generated data were: **user_id, amount, timestamp, ip_address**
- Transaction patterns were designed to avoid unrealistic sudden city jumps

# Models and Prediction

## Clustering & Feature Extraction

- We made clusters from the user's **unique_ips, amount_std and num_transactions** by applying **KMeans Clustering.**
- Also, we extracted previous 2 IP addresses for **each user and timestamp.**

## Markov Modelling

**First-order Markov:**

- Calculate **transition probabilities** from **prev_ip_prefix → current_ip_prefix.**

**Second-order Markov:**

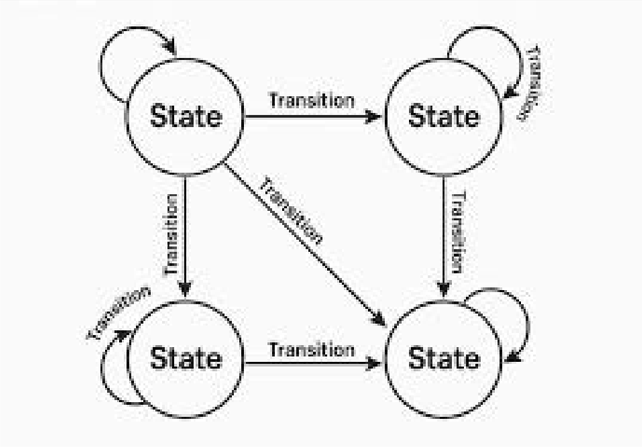
- Calculate **transition probabilities** from **(prev2_ip_prefix, prev_ip_prefix) → current_ip_prefix.**

## Data Processing

- Create **features**:
  - **Sequential**: prev2_ip_prefix, prev_ip_prefix.
  - **Temporal**: hour, weekday, amount.
  - **User behavior:** cluster.
  - **Markov**: first_order_prob, second_order_prob.
- Use **LabelEncoder** (fitted on all IPs) to convert **IP prefixes to numeric**.

## XGBoost Classifier

- Train XGBoost classifier to predict **next likely ip_prefix** for each user.
- For each user, take last **2 known IP prefixes + features.**
- Predict **next ip_prefix** using trained model.
- Use the mapping to **extract city names** from the ip_prefix predicted.




| | user_id | predicted_ip_prefix | predicted_city |
|---|---|---|---|
| 0 | user_001 | 64.1.152 | Bhusawal |
| 1 | user_002 | 199.245.22 | Silchar |
| 2 | user_003 | 42.83.205 | Thiruvananthapuram |
| 3 | user_004 | 51.165.50 | Haridwar |
| 4 | user_005 | 176.133.13 | Ambattur |
| .. | ... | ... | ... |
| 95 | user_096 | 208.154.87 | Indore |
| 96 | user_097 | 34.215.61 | Ajmer |
| 97 | user_098 | 34.215.61 | Ajmer |
| 98 | user_099 | 34.215.61 | Ajmer |
| 99 | user_100 | 67.65.112 | Kamarhati |

# Thank You!